

Reflection 反思范式 · 课程总结

封版日期：2026-08-04 (Windows) 定位：L8 追捕 / 校招闭环补齐。给阶段二 capstone 补上 **Planning-Acting-Reflection** 闭环缺的最后一环。对标书 Ch8 (持续进化 / self-improvement) 的入门形态。交付物： `reflect.py` (evaluator) + `agent.py run()` 接线 + `test_reflect.py` (门禁, 3 条)。

一、这节到底学了什么（一句话）

你 L8 造了会「想→做→看」的 loop；这节教它在**看完、要交卷之前，按红线自审一遍、不合格就重写一次**。工程上真正的难点不是「加个 critic」，而是**判断哪些该反思、给什么 rubric、如何不 over-correct、如何控成本、故障时怎么降级**。

二、核心概念（面试要能拆的三个层次）

「反思」是个大词，面试一问深就得能拆：

层次	是什么	代表	存不存 memory
(a) Self-Refine	对一次输出做 生成→批判→改写，不涉及任务成败	Madaan 2023	不存
(b) Reflexion	任务 失败后 （有 reward 信号）写「为什么失败/下次怎么改」， 存进 memory ，带着教训重试	Shinn 2023	存（语言化的 RL）
(c) Reflection node	agent 图里加 critic 节点 + 条件边判 accept/revise	LangGraph pattern	—

- **Reflexion 精髓**：不更新模型权重，更新的是 **context 里的经验** → 这就是它接到书 Ch8「持续进化」的原因。
- **本节做的 = (a)+(c)**：输出前加 critic，按 rubric 判 accept/revise。(b) 那种跨 attempt 存教训更重，讲清但没做（留 Ch8）。

反思三角色（Reflexion 解剖）

- **Actor (干活)** = `run()` loop。
- **Evaluator (挑错)** = 新加的 `reflect`。信号来源**越靠上越可靠**：
 1. 代码/规则（最可靠、零成本）——越权/超额这类**根本不该让 LLM 反思**，L8 已 hard-code。
 2. 真实环境反馈（最强）——测试是否通过、工具是否报错。
 3. LLM-as-judge（最灵活、有盲区）——判「代码判不了、但有对错」的话术层。
- **Self-reflection (改进指令)** = `reflect` 返回的 critique，被 `run()` 当反馈喂回 Actor 重写。

反直觉判断（本节最重要）：反思只留给代码判不了、但有对错的地方（话术是否泄露/越权/编造/答非所问）。能 hard-code 的继续 hard-code——**别把确定性红线换成概率性反思，那是倒退。**

三、工程判断（面试真正问的）

- 1. **反思不是免费的：**latency 可能翻倍、多烧 token、**可能 over-correct**（把对的改错）。
- 2. **什么时候值得反思：**输出**面向用户且不可撤回** + 有**明确 rubric** + **错误代价高**。中间步骤/代价低/无判据 → 别反思。
- 3. **触发方式：**全量（每次）vs 信号触发（有失败信号才反思，省成本，L8 stop_reason 回扣）。
- 4. **必须有次数上限：**否则反思→改→再反思会 thrashing（**L6 压缩 thrashing 回扣**）甚至死循环。本项目 `MAX_REFLECTIONS=1`。
- 5. **Evaluator 独立性：**同一模型「自己判自己」有盲区。缓解三招：换视角提问（「站在合规审查角度…」）、换更强模型当 judge、代码红线兜底。

四、落地到 capstone（施工记录）

落点选择（本身是设计）

- 落点 = `run()` 的**正常出口前**（`if not msg.tool_calls:` → 即将 return 那一刻）。
- **转人工（escalate 的 terminal）在工具循环里就提前 return 了，根本到不了这个分支** → 反思天然只审「正常给用户的话」，不审转人工。落点选择本身可讲。

reflect.py (evaluator)

- rubric = 把 `SYSTEM_PROMPT` 话术约束 + 三道安全线，**翻译成「质检员视角」**（主语从「你该怎么做」→「草稿若犯 X 则 revise」）。
- 结构化输出：`response_format=json_object`，返回 `{"verdict":"accept|revise","critique":"..."}`。
- **依赖注入：**`client` 当参数传，不在函数内 `get_client()` → 可 mock（L10）。
- **整段 try + fail-open：**API 调用 + 解析都进 try，任一失败 → `accept`。

agent.py run() 接线（3 处）

- 1. 循环前：`tool_trace=[]`（喂 reflect 判忠实性）、`reflections=0`、`MAX_REFLECTIONS=1`。
- 2. dispatch 循环内：每个 `result` append 进 `tool_trace`（**每个工具都要收**）。
- 3. 正常出口改写：未反思满 → 调 reflect；`revise` → append 草稿 + critique 反馈、`reflections+=1`、`continue` 重写；`accept` / 已满 → return。

实测证据（reflect 真在干活，不是摆设）

探针	草稿	结果
泄露归属	forbidden 却说「是别人的订单」	revise ，点名红线1
编造金额	工具返 199，草稿写 899	revise ，点名红线3

探针	草稿	结果
合规	忠实于工具返回	accept

端到端 6 场景全绿 (reflect 挂进 loop 不误伤好链路)。

五、这次课中的真实过程（踩坑与纠正）

- rubric 抄成了「演员台词」**：第一版几乎原样抄 `SYSTEM_PROMPT`（第二人称命令「你要调用 escalate 工具」）。但 reflect 是**质检员、不干活**——把演员指令给质检员，视角错位。纠正=翻译成「草稿**若**犯 X **则** revise」。
- 模型名写成 gpt-4o-mini**：DeepSeek 端点上没这模型，一跑就 400。judge 是「低频、质量关键」的活，改 `deepseek-v4-flash`（生产可上 pro）。
- fail-open 声明与代码打架**：口头选了 fail-open（对），但 `try` 只包了解析、API 调用在外面——而埋点问的正是「**模型宕机**」，那时 `create` 抛异常没接住 = 实际 fail-**closed**（崩溃式挡死）。改成整段 try 才自治。
- tool_trace.append 位置摆错**（头号短板「多部分/位置」的又一次变体）：放在 `for` 循环**外**，一轮多工具时只收到最后一个。挪进 for 循环内才每个都收。
- 两个埋点问答**：
 - fail-open 理由：质量层叠在**已执行的**三道 hardcode 安全线之上（安全早在工具里守死），自身故障不该拖垮 agent。
 - client 为何注入：**为了能 mock，让测试「拨网线还能过」**——把网络副作用作为依赖注入而非内部构造，函数才从「不可测」变「可测」。
- 「混血列表」澄清**：`run()` 里 append 纯文本 assistant 用 dict、带 tool_calls 用 SDK 对象，混着放**没问题**——坑不在混，而在「自己遍历 messages 时访问方式不统一（`m.content` vs `m["content"]`）」。这里只交给 `create()`，安全。

六、门禁三条（全过）

门禁	状态
① 环境可复现	uv / <code>uv run pytest</code>
② 至少 1 正 1 反 pytest 绿	13 passed : accept 正 / revise 反 / fail-open 兜底 (T5 把「模型宕机该 fail-open」钉死防回退)
③ 无调试残留	你来写 /TODO 清光, py_compile 通过, 无硬编码 key (client 注入)

七、Quiz 回顾 (5/5 达标)

- Q1** 已 hard-code 的退款阈值该不该改成 reflect 判？→ **不该**：可靠性来自约束，别用概率手段守确定性红线。
- Q2** Self-Refine vs Reflexion 本质区别？→ 一个不存 memory（纯质量精修）、一个失败后存教训重试（需 reward 信号）。
- Q3** 反思为何必须有上限？→ 防 thrashing/死循环/烧 token；**同类坑 L6 压缩阈值踩过**（这半题当时漏答，已补）。

- **Q4** 同模型自判的缺陷 + 两个缓解？ → 盲区（错答就是它生成的）；换模型/换视角/代码红线兜底。
 - **Q5** 哪个场景该反思？ → 面向用户的终态答复（不可撤回、代价高）该反思；内部中间结果（有后续兜底）不该。
-

八、研究锚点 & 下一步

- **一手论文**：Reflexion (Shinn et al. 2023)、Self-Refine (Madaan et al. 2023)。
 - **工程 pattern**：LangGraph reflection/reflexion tutorial（条件边 + critic 节点，可对比 L9 ADR）。
 - **书 Ch8**：真正的 self-improvement 会把反思教训沉淀成长期 memory / 微调数据 → Ch7/Ch8 研究向加分项。
 - **更重的版本（本节没做，留 Ch8）**：Reflexion 跨 attempt 存 memory——失败轨迹写成经验、下次带着重试。
-

一句话简历版：给客服 agent 补上 Planning-Acting-**Reflection** 闭环——用 LLM-as-judge 在答复发出前按红线自审，命中泄露/越权/编造则重写一次（有上限防 thrashing）；质检层 fail-open 且依赖注入可测，不拖垮已有的三道 hard-code 安全线。